Звіт до лабораторної роботи № 3

з дисципліни «Системне програмування»

Тема: Створення утиліти «DiskInfo»

1. Мета роботи

Ознайомитися з можливостями WinAPI щодо отримання інформації про дискову підсистему, файлові системи, системну пам'ять, користувацьке середовище та системні каталоги. Навчитися створювати системні утиліти моніторингу та інструменти спостереження за змінами у файловій системі на прикладі консольної утиліти DiskInfo.

2. Опис програми та її призначення

Утиліта DiskInfo — консольна програма для ОС Windows, написана мовою C++ з використанням лише функцій WinAPI (без сторонніх бібліотек). Програма послідовно виводить: перелік логічних дисків та їх статус; тип кожного диска з поясненням; параметри файлової системи кожного диска; обсяг зайнятого та вільного простору; стан оперативної та віртуальної пам'яті; ім'я комп'ютера та поточного користувача; шляхи до основних системних каталогів. Після виводу довідкової інформації утиліта запускає моніторинг обраного користувачем каталогу: усі події створення, видалення, перейменування та зміни файлів фіксуються з точним часом та записуються у log-файл diskinfo_watch.log.

Програма реалізована процедурно, у вигляді набору незалежних модулів (пара «.h / .cpp» на кожну функціональну область), що відповідає структурі, наведеній у завданні лабораторної роботи.

3. Структура програми

Програма складається із наступних файлів:

Файл	Призначення
main.cpp	Точка входу; послідовно викликає всі інформаційні модулі та запускає моніторинг каталогу.
DiskInfo.h / .cpp	Отримання списку логічних дисків, їх типу, інформації про файлову систему та вільний/зайнятий простір (завдання 1-4).
MemoryInfo.h / .cpp	Отримання інформації про фізичну та віртуальну пам'ять системи (завдання 5).
UserInfo.h / .cpp	Отримання імені комп'ютера, імені поточного користувача та системних каталогів (завдання 6-8).
Watcher.h / .cpp	Асинхронне відстеження змін у вибраному каталозі та запис подій у log-файл (завдання 9).
utils.h / .cpp	Допоміжні функції: форматування байтів у КБ/МБ/ГБ, форматування відсотків, отримання часової мітки.

4. Типи даних та глобальні змінні

Програма свідомо не використовує глобальних змінних: усі дані передаються через параметри та повертаються через значення функцій, що спрощує тестування модулів окремо один від одного та відповідає принципу мінімізації глобального стану. Єдиний виняток — стан завершення потоку моніторингу (running), який за своєю природою є розподіленим між двома потоками виконання і тому оголошений як локальна змінна функції WatchDirectory, що передається у другий потік за посиланням.

Основні типи даних, що використовуються в програмі, наведені в таблиці нижче.

Тип	Призначення
std::vector<std::string>	Список шляхів логічних дисків, отриманий від GetLogicalDriveStrings.
DWORD (unsigned long)	Коди помилок, прапори файлової системи, прапори відстеження змін (FILE_NOTIFY_CHANGE_*).
ULARGE_INTEGER	64-розрядні значення обсягу диска (загальний, вільний, доступний), що повертає GetDiskFreeSpaceEx.
MEMORYSTATUSEX	Структура з повною інформацією про фізичну та віртуальну пам'ять (GlobalMemoryStatusEx).
FILE_NOTIFY_INFORMATION	Структура з інформацією про одну подію зміни файлу (дія та ім'я файлу), яку повертає ReadDirectoryChangesW.
OVERLAPPED	Структура для асинхронного (неблокуючого) виконання ReadDirectoryChangesW разом з об'єктом-подією.
std::atomic<bool>	Прапорець running, яким потік очікування клавіші ENTER безпечно сигналізує основному потоку про завершення моніторингу.

5. Методи та прийоми, використані при вирішенні задачі

1. Модульна декомпозиція задачі: кожне завдання лабораторної роботи (1-9) винесено в окремий заголовний та cpp-файл, що спрощує читання коду та відповідає вимогам до оформлення.
2. Використання ANSI-варіантів функцій WinAPI (суфікс «A», напр. GetVolumeInformationA), оскільки шляхи та імена в даній роботі оброблюються як однобайтові рядки std::string.

3. Обробка кодів завершення: після кожного виклику WinAPI-функції перевіряється повернене значення (BOOL/HANDLE), і в разі помилки виводиться код через GetLastError().
4. Асинхронний (перекриваний, overlapped) ввід-вивід для моніторингу каталогу: ReadDirectoryChangesW викликається з структурою OVERLAPPED та очікуванням через WaitForMultipleObjects, що дозволяє одночасно очікувати як подію файлової системи, так і сигнал завершення від користувача.
5. Багатопотоковість (std::thread): окремий потік очікує натискання клавіші ENTER у консолі, не блокуючи основний потік, який очікує подій файлової системи.
6. Виділення допоміжних (utility) функцій форматування (FormatBytes, FormatPercent, GetTimestamp) в окремий модуль для уникнення дублювання коду.
7. Ведення log-файлу: кожна подія моніторингу каталогу записується у текстовий файл із часовою міткою, типом події та ім'ям файлу (режим дозапису std::ios::app, щоб не втрачати історію попередніх сеансів).

6. Розбір використаних функцій WinAPI

Нижче наведено розбір основних функцій WinAPI, використаних у програмі, за шаблоном: тип поверненого значення; що воно означає; параметри та їх призначення; що виконує функція; особливості використання. Для функцій, що приймають прапорці-константи, додатково наведено перелік споріднених констант із коротким поясненням.

6.1. GetLogicalDriveStringsA (task 1 — перелік логічних дисків)

Синтаксис	DWORD GetLogicalDriveStringsA(DWORD nBufferLength, LPSTR lpBuffer);
Тип повернення	DWORD — довжина записаних даних у символах (без кінцевого нуля) або 0 у разі помилки.
Параметри	nBufferLength — розмір буфера lpBuffer у символах; lpBuffer — буфер, куди записується перелік дисків.
Що виконує	Заповнює буфер послідовністю рядків виду "C:\\\0D:\\\0", що завершується подвійним нульовим символом; кожен рядок — корінь одного логічного диска.
Особливості	На відміну від GetLogicalDrives (яка повертає бітову маску 0-25), одразу дає готові рядки шляхів, зручні для подальших викликів GetDriveType, GetVolumeInformation тощо.

6.2. GetDriveTypeA (task 2 — тип диска)

Синтаксис	UINT GetDriveTypeA(LPCSTR lpRootPathName);
Тип повернення	UINT — одна з констант DRIVE_*, що визначає тип пристрою зберігання.
Параметри	lpRootPathName — кореневий шлях диска, напр. "C:\\".
Що виконує	Визначає фізичний тип носія, змонтованого за вказаним шляхом.
Особливості	Не потребує прав доступу до вмісту диска; працює навіть якщо диск не має файлової системи.

Константа	Значення / пояснення
DRIVE_UNKNOWN	Тип диска визначити неможливо.
DRIVE_NO_ROOT_DIR	Некоректний кореневий шлях (том відсутній).
DRIVE_REMOVABLE	Змінний носій: USB flash, SD-картка.
DRIVE_FIXED	Незмінний внутрішній накопичувач: HDD або SSD.
DRIVE_REMOTE	Мережевий диск (спільний ресурс).
DRIVE_CDROM	Привід оптичних дисків CD/DVD.
DRIVE_RAMDISK	Віртуальний диск в оперативній пам'яті.

6.3. GetVolumeInformationA (task 3 — файлова система)

Синтаксис	BOOL GetVolumeInformationA(LPCSTR lpRootPathName, LPSTR lpVolumeNameBuffer, DWORD nVolumeNameSize, LPDWORD lpVolumeSerialNumber, LPDWORD lpMaximumComponentLength, LPDWORD lpFileSystemFlags, LPSTR lpFileSystemNameBuffer, DWORD nFileSystemNameSize);
Тип повернення	BOOL — TRUE у разі успіху, FALSE у разі помилки (деталі через GetLastError).
Параметри	lpRootPathName — корінь тому; lpVolumeNameBuffer/nVolumeNameSize — буфер під мітку тому; lpVolumeSerialNumber — серійний номер; lpMaximumComponentLength — максимальна довжина імені файлу; lpFileSystemFlags — бітова маска можливостей файлової системи; lpFileSystemNameBuffer/nFileSystemNameSize — буфер під назву файлової системи (NTFS, FAT32

	тощо).
Що виконує	Зчитує метадані тому: мітку, серійний номер, обмеження на довжину імені файлу, назву та прапори підтримуваних можливостей файлової системи.
Особливості	Для приводів CD-ROM без диска або недоступних мережевих ресурсів виклик може завершитись помилкою або тривалою затримкою, тому в програмі такі диски попередньо відсіюються за результатом GetDriveType.

Константа (lpFileSystemFlags)	Пояснення
FILE_FILE_COMPRESSION	Файлова система підтримує стиснення окремих файлів.
FILE_SUPPORTS_ENCRYPTION	Підтримка шифрувальної файлової системи (EFS).
FILE_PERSISTENT_ACLS	Підтримка списків контролю доступу (ACL).
FILE_SUPPORTS_TRANSACTIONS	Підтримка транзакційних операцій (журналювання NTFS).
FILE_CASE_SENSITIVE_SEARCH	Пошук з урахуванням регістру символів імені файлу.
FILE_UNICODE_ON_DISK	Імена файлів зберігаються у форматі Unicode.
FILE_SUPPORTS_SPARSE_FILES	Підтримка розріджених (sparse) файлів.

6.4. GetDiskFreeSpaceExA (task 4 — вільний/зайнятий простір)

Синтаксис	BOOL GetDiskFreeSpaceExA(LPCSTR lpDirectoryName, PULARGE_INTEGER lpFreeBytesAvailable, PULARGE_INTEGER lpTotalNumberOfBytes, PULARGE_INTEGER lpTotalNumberOfFreeBytes);
Тип повернення	BOOL — TRUE у разі успіху, FALSE у разі помилки.
Параметри	lpDirectoryName — шлях на диску; lpFreeBytesAvailable — простір, доступний виклику (з урахуванням квот); lpTotalNumberOfBytes — загальний обсяг тому; lpTotalNumberOfFreeBytes — весь вільний простір тому.
Що виконує	Повертає 64-розрядні значення обсягу диска, з яких обчислюються зайнятий простір та відсоток завантаження.
Особливості	На відміну від застарілої GetDiskFreeSpace, коректно працює з томами понад 2 ГБ завдяки типу ULARGE_INTEGER.

6.5. GlobalMemoryStatusEx (task 5 — системна пам'ять)

Синтаксис	BOOL GlobalMemoryStatusEx(LPMEMORYSTATUSEX lpBuffer);
Тип повернення	BOOL — TRUE у разі успіху, FALSE у разі помилки.
Параметри	lpBuffer — вказівник на структуру MEMORYSTATUSEX; перед викликом обов'язково встановлюється поле dwLength = sizeof(MEMORYSTATUSEX).
Що виконує	Заповнює структуру даними про завантаження ОЗП (%), обсяг фізичної та вільної пам'яті, обсяг файлу підкачки та віртуальної пам'яті.
Особливості	Обрана замість застарілої 32-розрядної GlobalMemoryStatus, оскільки коректно відображає обсяги пам'яті понад 4 ГБ.

6.6. GetComputerNameA / GetUserNameA (task 6-7)

Синтаксис	BOOL GetComputerNameA(LPSTR lpBuffer, LPDWORD nSize); BOOL GetUserNameA(LPSTR lpBuffer, LPDWORD lpnSize);
Тип повернення	BOOL — TRUE у разі успіху, FALSE у разі помилки.
Параметри	lpBuffer — буфер під рядок результату; nSize/lpnSize — на вході розмір буфера, на виході фактична довжина рядка.
Що виконує	GetComputerNameA повертає NetBIOS-ім'я комп'ютера; GetUserNameA — ім'я користувача, від імені якого виконується потік.
Особливості	Розмір буфера для імені комп'ютера обмежений константою MAX_COMPUTERNAME_LENGTH + 1.

6.7. GetSystemDirectoryA / GetWindowsDirectoryA / GetTempPathA / GetCurrentDirectoryA (task 8)

Тип повернення	DWORD — довжина отриманого шляху в символах (0 у разі помилки).
Параметри	Буфер під шлях та його розмір (для GetTempPathA порядок параметрів зворотний — спочатку розмір буфера).
Що виконує	Повертають відповідно шлях до каталогу System32, каталогу Windows, тимчасового каталогу користувача та поточного робочого каталогу процесу.
Особливості	Усі шляхи обмежені константою MAX_PATH (260 символів) у застосованому в роботі ANSI-варіанті API.

6.8. ReadDirectoryChangesW (task 9 — моніторинг каталогу, рівень «наступний»)

Синтаксис	BOOL ReadDirectoryChangesW(HANDLE hDirectory, LPVOID lpBuffer, DWORD nBufferLength, BOOL bWatchSubtree, DWORD dwNotifyFilter, LPDWORD lpBytesReturned, LPOVERLAPPED lpOverlapped, LPOVERLAPPED_COMPLETION_ROUTINE lpCompletionRoutine);
Тип повернення	BOOL — TRUE, якщо запит на очікування подій успішно поставлено в чергу, FALSE у разі помилки.
Параметри	hDirectory — дескриптор каталогу (відкритий з прапорцем FILE_FLAG_BACKUP_SEMANTICS); lpBuffer/nBufferLength — буфер під записи FILE_NOTIFY_INFORMATION; bWatchSubtree — чи стежити за підкаталогами; dwNotifyFilter — маска типів подій; lpBytesReturned — використовується лише в синхронному режимі; lpOverlapped — структура для асинхронного очікування.
Що виконує	Асинхронно очікує зміни у вказаному каталозі; після завершення операції буфер містить один або декілька записів FILE_NOTIFY_INFORMATION з кодом дії та іменем зміненого файлу.
Особливості	На відміну від FindFirstChangeNotification (яка лише сигналізує «щось змінилося», вимагаючи повторного сканування каталогу), одразу повертає ім'я файлу та тип події, що й відповідає «наступному рівню» виконання завдання 9.

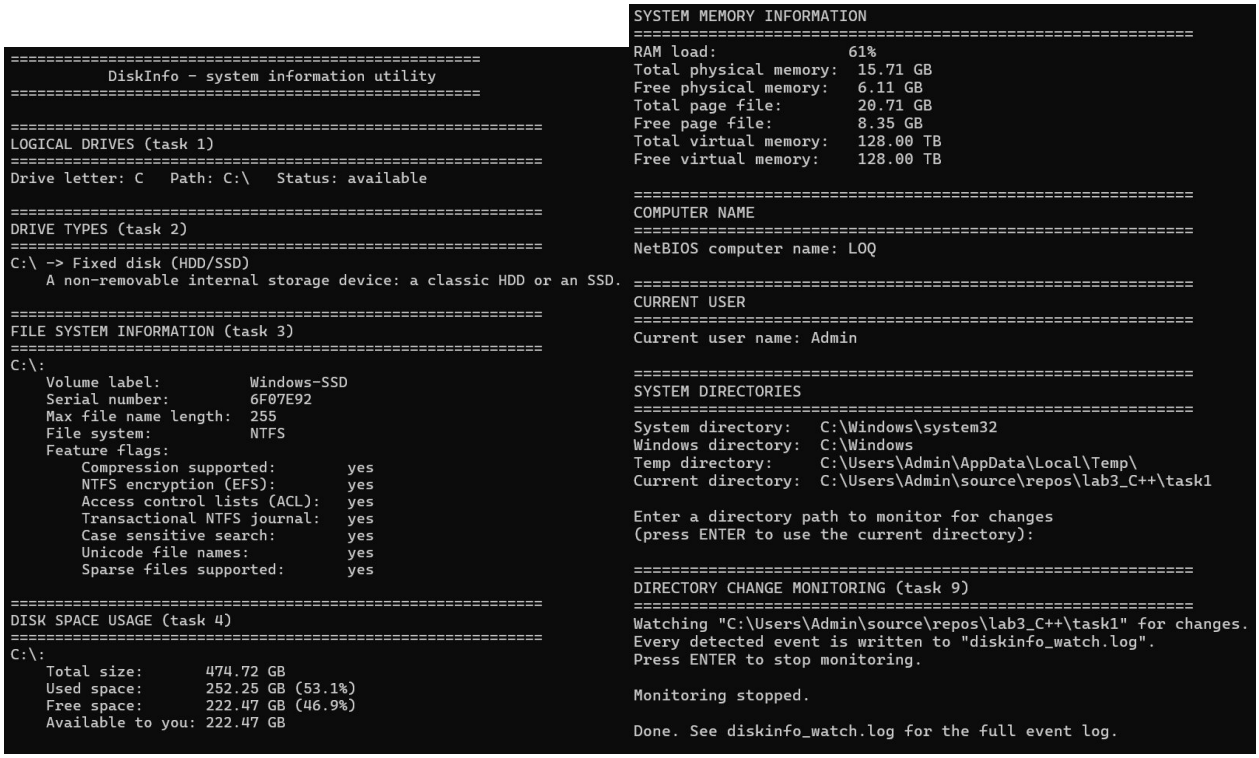
Константа (dwNotifyFilter)	Пояснення
FILE_NOTIFY_CHANGE_FILE_NAME	Створення, видалення або перейменування файлу.
FILE_NOTIFY_CHANGE_LAST_WRITE	Зміна вмісту файлу (запис даних).
FILE_NOTIFY_CHANGE_ATTRIBUTES	Зміна атрибутів файлу (напр. «тільки читання»).
FILE_NOTIFY_CHANGE_DIR_NAME	Створення або видалення підкаталогу (не використано в даній роботі).
FILE_NOTIFY_CHANGE_SIZE	Зміна розміру файлу (не використано в даній роботі).

Константа (FILE_NOTIFY_INFORMATION::Action)	Пояснення
FILE_ACTION_ADDED	Файл додано до каталогу.
FILE_ACTION_REMOVED	Файл видалено з каталогу.
FILE_ACTION_MODIFIED	Файл змінено (запис, зміна атрибутів тощо).
FILE_ACTION_RENAMED_OLD_NAME	Стара назва перейменованого файлу.
FILE_ACTION_RENAMED_NEW_NAME	Нова назва перейменованого файлу.

7. Компіляція та перевірка працездатності

Програма зібрана та запущена в середовищі Visual Studio на ОС Windows. Для збірки створено порожній консольний проєкт C++, до якого додано всі файли утиліти: заголовні файли DiskInfo.h, MemoryInfo.h, UserInfo.h, Watcher.h, utils.h та файли реалізації DiskInfo.cpp, main.cpp, MemoryInfo.cpp, UserInfo.cpp, utils.cpp, Watcher.cpp. Збірка проєкту пройшла без помилок компіляції, програму запущено через Ctrl+F5 (запуск без налагодження). Нижче наведено результат виконання програми.

Результат виконання програми:



8. Висновок

У ході виконання лабораторної роботи розроблено консольну утиліту DiskInfo мовою C++, яка засобами WinAPI (без сторонніх бібліотек) отримує інформацію про логічні диски, їх тип, файлову систему та обсяг вільного/зайнятого простору, стан оперативної та віртуальної пам'яті, ім'я комп'ютера й поточного користувача, а також основні системні каталоги. Реалізовано відстеження змін у файловій системі підвищеного («наступного») рівня складності — через асинхронний виклик ReadDirectoryChangesW із веденням текстового log-файлу подій. У процесі роботи закріплено практичні навички використання основних груп функцій WinAPI для системного моніторингу, а також прийомів асинхронного введення-виведення та багатопотокової обробки консольного вводу.